

DataVortex :: Round 2 SQL Query Logic & Walkthrough

Team Name: Fernandoisfasterthanyou

Email: pathakuntlabhaskarreddy@gmail.com

Event: DataVortex — Social Engine Forensic & Analytics Suite

Companion File: Round2_SQL_Queries_Only.pdf

A Note on How We Approached These Queries

When working with corrupted real-world data, writing SQL isn't just about getting a query to run without syntax errors. It's about knowing what traps are hiding inside the rows and writing code that reflects what actually happened on the platform.

The raw data we recovered had missing platform tags, negative likes caused by sign-bit flips, and unescaped HTML garbage. If you query that blindly, your aggregations lie to you.

Below is our walkthrough of the thinking, the edge cases we had to watch out for, and how each query works from E1 through H6.

Easy Level Queries (E1 – E5)

E1 — Finding the Most Popular Platform

The goal here was simple: find out which social media platform had the highest total number of published posts.

On paper, that's just grouping by platform and counting rows. But there's an immediate catch in the dataset: roughly 15% of all records lost their platform tag during the intake crash, showing up as `NULL` or empty strings. If you just run a blind `GROUP BY`, your top "platform" could literally be empty space.

To fix that, we filtered out blank and null platform values upfront with `WHERE platform IS NOT NULL AND TRIM(platform) ≠ ''`, counted the rows, ordered descending, and grabbed the top row (`LIMIT 1`).

What the result showed: Facebook came out on top with 2,074 posts. What's interesting is how close the other platforms were—YouTube had 2,071, Twitter had 2,060, and Reddit had 2,028. Each platform accounted for almost exactly 20% of the traffic, which tells us the social engine was distributing intake evenly across queues before it broke.

E2 — Tracking Down the Top 10 Most Engaged Posts

We wanted to find the 10 biggest viral hit posts on the platform based on total engagement, which the challenge defined as likes + shares + comments.

The tricky part in SQL is how `NULL` behaves in addition. In standard SQL math, `NULL + 1,000 + 500` doesn't give you 1,500—it returns `NULL`. If a post had 2,000 shares and 1,000 comments but its likes field was blank, simple addition would wipe the entire engagement score to null and drop it from the leaderboard.

Since the prompt told us to ignore posts where likes were missing, we added an explicit `WHERE likes IS NOT NULL`, added the three numbers together, sorted descending, and took the top 10.

What the result showed: The highest total engagement on the platform capped out at 7,893 interactions (held by post `ycjj5zzt7mvx` on Instagram with 4,983 likes, 1,919 shares, and 991 comments). Across all top 10 posts, likes hovered near the 5,000 ceiling, shares near 2,000, and comments near 1,000.

E3 — Comparing Average Engagement Across Platforms

Here, we wanted to see which platform gives creators the best return on average by calculating the mean likes, shares, comments, and total engagement per post for each network.

Because raw likes had missing entries, we used `COALESCE(likes_cleaned, likes)` so our calculations could leverage the restored likes values from our CatBoost imputation model without biasing the sample downward. We grouped by platform, averaged each metric with `AVG()`, rounded to two decimal places, and sorted by overall average engagement.

What the result showed: Instagram took #1 overall, averaging 4,043.6 interactions per post, mainly because it pulled significantly higher shares (1,040 on average). But Reddit dominated on discussion depth, pulling in the highest average comments (511.2 per post). Creators get more reach on Instagram, but more conversation on Reddit.

E4 — Catching High-Share, Low-Like Posts (Viral Friction)

This query hunts for posts that people shared like crazy (over 1,500 shares) but barely liked at all (fewer than 500 likes). In social media, that pattern usually signals outrage, breaking news, or heated debate where people want others to see it, but hitting the "like" button feels wrong.

The trap here was in the corrupted data: 525 posts had negative like numbers due to a signed-integer bit flip (like `-4812`). If you write `WHERE likes < 500` without caution, a post with `-4812` (which actually had 4,812 likes!) would falsely show up here. We explicitly added `likes ≥ 0` to ensure we only captured real, low-like posts.

What the result showed: We found extreme outliers like post `1it2hyqg0v0l` on Facebook: only 14 likes, but 1,990 shares and 769 comments. That's a share-to-like ratio of over 140 to 1, a textbook example of high-friction social engagement.

E5 — Spotting Macro-Influencers (40k+ Followers)

The goal was to list every user on the platform who has built an audience of more than 40,000 followers, along with their location and primary language.

This was a clean filter on the `users` table: `WHERE follower_count > 40000`, sorted descending so the biggest creators appear first.

What the result showed: Follower counts in this platform top out just under 50,000 (the biggest account is `user_3o7w66o2` in Berlin with 49,944 followers). The user base is thoroughly international, with top creators based in Tokyo, Chicago, Shanghai, Barcelona, and Paris.

Medium Level Queries (M1 – M5)

M1 — Mapping Engagement by Creator Location

We wanted to connect the two datasets together to see which global cities generate the most total engagement on the platform.

Posts only store `user_id`, while the city and country live in the `users` table. So we did an `INNER JOIN` between `users` and `posts` on `user_id`, grouped by `u.location`, counted each city's posts, and summed up the total interactions. We used `COALESCE` on likes so that any missing like count defaulted to 0 rather than erasing the sum.

What the result showed: Los Angeles led the world with 1.85 million interactions across 459 posts, with Munich (1.80M) and Shanghai (1.79M) right behind it. The total engagement in each city mapped almost directly to how many posts came out of that city, showing consistent per-post engagement worldwide.

M2 — Testing the Follower Myth (Do Big Accounts Get More Engagement?)

The question everyone in marketing asks: *Does having more followers actually get you more engagement per post?*

To test this cleanly, we split creators right down the middle at the 25,000 follower median into two groups: high-follower accounts ($\geq 25,000$) and low-follower accounts ($< 25,000$). We used a `CASE` statement to bucket the users, joined their posts, and calculated the average engagement per post for both groups.

What the result showed:

- * Accounts with 25,000+ followers averaged **3,997.41** interactions per post.
- * Accounts with under 25,000 followers averaged **3,995.67** interactions per post.

The difference was just **1.74 interactions per post**. This was one of the biggest insights in the whole project: follower count has essentially zero impact on engagement. The platform's algorithm pushes content based on post quality and watch time, not legacy audience size.

M3 — Identifying the Most Active Power Creators

We wanted to see who the hardest-working creators were on the platform—the top 10 users who published the highest number of posts—and inspect how many followers they had and how much total engagement they generated.

We grouped by the user's details (`user_id` , `follower_count` , `location`), counted their posts, summed their total engagement, and sorted primarily by post count descending, using total engagement as a tie-breaker.

What the result showed: `user_zqv2vrf5` in Vancouver was the most prolific creator with 22 posts and ~86,000 total interactions. What stood out was that two creators with under 2,000 followers (`user_uerv85na` with 1,824 followers; `user_hdas0iau` with 1,620 followers) were in the top 10 most active users, each putting out 16 posts and pulling over 66,000 interactions.

M4 — Which Platform Works Best for Big Creators?

When macro-influencers (creators with at least 30,000 followers) post, which specific social network rewards them with the highest average engagement per post?

We filtered for users with `follower_count` $\geq$ 30000 , joined their posts, grouped by platform, and calculated their average engagement.

What the result showed: Instagram was the clear winner for large creators, delivering **4,109.61 interactions per post**—more than 150 interactions higher than YouTube (3,953.80). Visual and aesthetic formats on Instagram seem to give established creators a distinct amplification edge.

M5 — Flagging Bot-Like or Suspicious Sharing Behavior

Here, we looked for posts where the number of shares alone was greater than the likes and comments combined.

On social media, liking something is effortless (one tap), commenting takes typing, and sharing takes deliberate intent to broadcast to your own feed. So when shares alone dwarf both likes and comments put together, it points to either automated bot amplification or extreme viral controversy.

We computed `likes_plus_comments` , filtered for posts where `shares` $>$ `likes_plus_comments` , calculated the exact difference (`share_excess`), and sorted by shares descending.

What the result showed: We caught posts like `u1aa801qvxeu` , where shares hit 1,992 while likes and comments combined were only 468—a massive excess of +1,524 shares. In a platform security audit, these 20 posts would be the first places to look for coordinated sharing rings.

Hard Level Queries (H1 – H6)

H1 — Spotting Creators Outperforming the Platform Average by 2×

We wanted to find the top-performing creators whose total accumulated engagement was **more than double ($> 2\times$) the platform-wide average for a creator**.

You can't do this in a simple single-level query because you first have to calculate what the average creator's total engagement even is. We broke the problem into two Common Table Expressions (CTEs):

1. `UserTotalStats` : Sums total engagement and post counts for every user.
2. `OverallUserAvg` : Takes the average of those user totals across the platform.

Then in the final `SELECT`, we crossed the two tables and filtered for creators whose total was greater than `2 * o.avg_user_total`.

What the result showed: The average creator accumulated ~31,980 total interactions across their posts. The top 10 creators doubled that easily, pulling between 66,000 and 85,967 interactions. `user_2ytzut7i` in Mumbai stood out with an average of 4,906 interactions on every post they made.

H2 — Creating Local Podium Leaderboards (Top 3 per City)

For every city represented in the database, we wanted to rank creators by total engagement and return the top 3 podium finishers for each location.

If you just write `ORDER BY total_engagement DESC LIMIT 3`, you only get 3 people for the entire world. To restart the ranking counter at #1 for each city, we used the SQL window function:

`DENSE_RANK() OVER (PARTITION BY location ORDER BY total_engagement DESC)`.

The `PARTITION BY location` clause splits the ranking into local buckets, and the outer query simply filters `WHERE location_rank ≤ 3`.

What the result showed: We got a localized top-3 ranking for every city worldwide. For example, in Barcelona, `user_nfo3ih5u` took #1 with 79.4k engagement; in Chicago, `user_ujllj1n7` took #1 with 72.2k; and in Berlin, `user_m5z8a3sg` won gold with 56.5k.

H3 — Identifying True Breakout Posts (2× Their Own Platform Average)

Every social network has its own baseline. This query searches for breakout viral posts that generated **at least double ($\geq 2\times$) their specific platform's average engagement**.

Comparing a tweet to an overall platform-wide average wouldn't be fair if Twitter had a lower baseline than Instagram. Each post needed to be judged against its own network.

We calculated each platform's baseline average in a CTE called `PlatformAvg`, calculated each post's engagement in `PostStats`, joined them on `platform`, and filtered for posts where `post_engagement ≥ 2 * platform_avg_engagement`.

What the result showed: Baseline averages across platforms sat around ~3,400 interactions, so a post needed roughly 6,800+ interactions to qualify. The top breakout posts reached 7,610 to 7,893 interactions—over **2.3 times** their platform's typical engagement.

H4 — The Giant Slayers (Under 5,000 Followers in the Top 10%)

This query hunts for underdogs: creators with tiny followings (**fewer than 5,000 followers**) who still managed to pull in so much engagement that they ranked in the **top 10% (90th percentile)** of all creators on the platform.

To find percentiles in SQL, we used the window function `PERCENT_RANK() OVER (ORDER BY total_engagement)`. This assigns every creator a score between \$0.00\$ and \$1.00\$. A score of \$0.90\$ means that creator beat 90% of the user base. We then filtered for `follower_count < 5000` AND `engagement_percentile ≥ 0.90`.

What the result showed: `user_67hyf45u` in Vancouver has **only 898 followers**, but racked up 51,406 interactions, putting them in the **93.2nd percentile**. Even more extreme, `user_uerv85na` in Rome has just 1,824 followers and reached 71,836 interactions—beating **99.73% of all creators on the platform**. This proves once again that great content beats follower count.

H5 — Building an Anomaly Audit Log on the Corrupted Data

We wanted to scan the raw corrupted table (`posts_corrupted`) and build an audit report that classified every corrupted post by its exact flaw: Negative Likes, Missing Platform, Missing Text, or Injected HTML Tags.

The big architectural trap here is that a single post can have multiple flaws (for example, negative likes *and* injected HTML). If you write a `CASE ... WHEN ... ELSE` statement, SQL stops evaluating as soon as the first condition is met, hiding the second flaw!

To catch every flaw, we used **UNION ALL across four independent queries**—one for negative likes, one for missing platforms, one for blank text, and one for HTML markup (`<div>` , `<br>` , `&`).

What the result showed: This produced an audit log where multi-corrupted posts correctly appear under every defect category they triggered (for example, post `0066x8nnmouc` appears under both Missing Platform and HTML Entities). This gave our data restoration pipeline a checklist of what needed to be fixed.

H6 — Spotting Suspicious High-Impact Accounts

Finally, we combined multiple signals to identify suspicious creator accounts: users who have small followings (**\$< 10,000\$ followers**), maintain **above-average engagement per post**, and have published at least one post where **shares beat likes**.

We calculated the platform's average engagement in `OverallMetric` , aggregated user metrics and counted suspicious share-heavy posts in `UserMetrics` , filtered for the three criteria, and ranked the

results with `DENSE_RANK()` .

What the result showed: `user_uerv85na` in Rome took the #1 spot: with only 1,824 followers, they averaged an astounding 4,489 interactions across 16 posts and had multiple posts where shares exceeded likes. In a platform integrity audit, these 10 accounts are the exact profiles you'd inspect to see if they're authentic viral creators or using automated share pods.

Final Thoughts

Looking across all 16 queries, the story of the DataVortex Social Engine becomes clear:

1. **The algorithm is democratic:** Followers don't guarantee reach, and micro-creators regularly outperform macro-influencers.
 2. **Each platform has a personality:** Instagram drives the highest total shares, while Reddit drives deep, authentic comment conversations.
 3. **Data cleaning matters:** Blind queries on corrupted numbers produce misleading results, but careful, defensive SQL reveals the real story beneath the noise.
-

Written by Team Fernandoisfasterthanyou (`pathakuntlabhaskarreddy@gmail.com`)